```
# realtime_training_pipeline.py
import torch
import torch.nn as nn
from transformers import AutoTokenizer,
AutoModelForCausalLM
import numpy as np
from typing import List, Dict, Tuple
import json
import time

class RealtimeInteractionDataset:
    """Dataset simulating real-time
multi-stream interaction"""

    def __init__(self, vocab_size=50257,
num_streams=4, max_length=2048):
        self.vocab_size = vocab_size
        self.num_streams = num_streams
        self.max_length = max_length

        # Stream roles (matching AICO
architecture)
        self.stream_roles = [
            "controller_stream",      #
Compilation thoughts
            "introspection_stream",   # Safety
audits
```

```
            "nervous_system_stream", #
Hardware state
            "comms_stream"           #
Dialogue shaping
        ]

        # Real-time phenomena to simulate
        self.phenomena = {
            "interruption": 0.3,      # User
interrupts mid-response
            "parallel_input": 0.4,    #
Multiple inputs arrive simultaneously
            "context_overlap": 0.5,   # New
context during processing
            "latency_deadline": 0.6,  # Must
respond within timeframe
            "stream_sync": 0.7        #
Streams must stay synchronized
        }

    def generate_example(self) -> Dict:
        """Generate a training example with
real-time characteristics"""

        # 1. Generate base dialogue context
        dialogue =
self._generate_dialogue_context()
```

```
        # 2. Add parallel streams
        streams = {}
        for role in self.stream_roles:
            streams[role] =
self._generate_stream_content(role, dialogue)

        # 3. Introduce real-time events
        events = []

        # Random interruption
        if np.random.random() <
self.phenomena["interruption"]:
            interruption_point =
np.random.randint(10, len(dialogue["tokens"])
- 5)
            events.append({
                "type": "interruption",
                "position":
interruption_point,
                "new_input":
self._generate_interruption()
            })

        # Parallel inputs
        if np.random.random() <
self.phenomena["parallel_input"]:
```

```
            num_parallel =
np.random.randint(2, 4)
            events.append({
                "type": "parallel_inputs",
                "inputs":
[self._generate_parallel_input() for _ in
range(num_parallel)]
            })

        # Latency deadline
        if np.random.random() <
self.phenomena["latency_deadline"]:
            events.append({
                "type": "latency_deadline",
                "max_tokens":
np.random.randint(50, 200),  # Must respond
within N tokens
                "time_ms":
np.random.randint(1000, 3000)   # Simulated
time budget
            })

        # 4. Create target: how streams should
coordinate
        target =
self._create_coordination_target(streams,
events)
```

```
        return {
            "dialogue": dialogue,
            "streams": streams,
            "events": events,
            "target": target,
            "metadata": {
                "num_tokens":
len(dialogue["tokens"]),
                "num_streams": len(streams),
                "has_interruption":
any(e["type"] == "interruption" for e in
events)
            }
        }

    def _generate_dialogue_context(self) ->
Dict:
        """Generate a simulated dialogue"""
        topics = ["hardware_driver",
"temperature_control", "user_query",
                 "system_diagnostic",
"creative_writing"]
        topic = np.random.choice(topics)

        # Generate tokens (simplified)
        tokens =
```

```
list(range(np.random.randint(50, 200)))

        return {
            "topic": topic,
            "tokens": tokens,
            "speakers": ["user", "assistant"]
* 5
        }

    def _generate_stream_content(self, role:
str, dialogue: Dict) -> List:
        """Generate content for each stream
role"""

        if role == "controller_stream":
            # Compilation thoughts
            return [
                {"type": "task_graph",
"content": "decomposing intent..."},
                {"type": "optimization",
"content": "JIT compiling driver..."},
                {"type": "memory_layout",
"content": "allocating buffers..."}
            ]

        elif role == "introspection_stream":
            # Safety audits
```

```
            return [
                {"type": "dcx_check", "score": 
np.random.uniform(0.1, 0.9)},
                {"type": "safety_audit", 
"passed": np.random.choice([True, False])},
                {"type": "drift_detection", 
"detected": np.random.random() > 0.8}
            ]

        elif role == "nervous_system_stream":
            # Hardware state
            return [
                {"type": "thermal", "temp": 
np.random.uniform(60, 85)},
                {"type": "utilization", "cpu": 
np.random.uniform(0.3, 0.9)},
                {"type": "latency", "p50_ms": 
np.random.uniform(1, 100)}
            ]

        else:  # comms_stream
            # Dialogue shaping
            return [
                {"type": 
"response_generation", "stage": "drafting"},
                {"type": "tone_adjustment", 
"valence": np.random.uniform(-0.5, 0.5)},
```

```
                {"type":
"context_integration", "new_context": True}
            ]

    def _generate_interruption(self) -> Dict:
        """Generate an interruption"""
        interruption_types = [
            "user_correction",
            "new_priority",
            "clarification_request",
            "emergency_override"
        ]

        return {
            "type":
np.random.choice(interruption_types),
            "urgency": np.random.uniform(0.5,
1.0),
            "content": "New information
arrived while you were processing."
        }

    def _create_coordination_target(self,
streams: Dict, events: List) -> Dict:
        """Create target coordination
pattern"""
```

```
        target = {
            "big_boat_summary": "",  # What
goes to central coordination
            "stream_coordination": {},
            "latency_budget": {},
            "interruption_handling": {}
        }

        # Generate ideal coordination
        for stream_name, content in
streams.items():
            # Summarize for big boat stream
            if stream_name ==
"controller_stream":
                target["big_boat_summary"] +=
"[CONTROLLER] Task decomposed, compiling. "
            elif stream_name ==
"comms_stream":
                target["big_boat_summary"] +=
"[COMMS] Response drafted, adjusting tone. "

            # Stream-specific coordination
targets

target["stream_coordination"][stream_name] = {
                "should_update": True,
                "update_frequency":
```

```
np.random.choice(["continuous", "periodic",
"event"]),
                "priority":
np.random.randint(1, 5)
            }

        # Handle events in targets
        for event in events:
            if event["type"] ==
"interruption":

target["interruption_handling"] = {
                    "action": "merge_context",
                    "priority_boost": 0.3,
                    "context_preservation":
0.8
                }
            elif event["type"] ==
"latency_deadline":
                target["latency_budget"] = {
                    "max_tokens":
event["max_tokens"],
                    "stream_prioritization":
["comms_stream", "controller_stream",
"introspection_stream"],
                    "allow_partial": True
                }
```

```
        return target

class ParallelCognitionTransformer(nn.Module):
    """Transformer architecture for parallel
real-time cognition"""

    def __init__(self, vocab_size=50257,
d_model=768, n_heads=12, n_layers=12,
                 num_streams=4,
max_length=2048):
        super().__init__()

        self.num_streams = num_streams
        self.d_model = d_model

        # Shared embedding layer
        self.token_embedding =
nn.Embedding(vocab_size, d_model)

        # Stream-specific embeddings (like
different "minds")
        self.stream_embeddings =
nn.Embedding(num_streams, d_model)

        # Main transformer with modified
attention
```

```
        encoder_layer =
nn.TransformerEncoderLayer(
            d_model=d_model,
            nhead=n_heads,
            dim_feedforward=d_model * 4,
            dropout=0.1,
            batch_first=True
        )
        self.transformer =
nn.TransformerEncoder(encoder_layer,
num_layers=n_layers)

        # Parallel attention heads (one per
stream)
        self.stream_attentions =
nn.ModuleList([
            nn.MultiheadAttention(d_model,
n_heads, batch_first=True)
            for _ in range(num_streams)
        ])

        # Cross-stream attention (for
coordination)
        self.cross_stream_attention =
nn.MultiheadAttention(
            d_model, n_heads, batch_first=True
        )
```

```
        # Output heads per stream
        self.stream_outputs = nn.ModuleList([
            nn.Sequential(
                nn.Linear(d_model, d_model *
2),
                nn.GELU(),
                nn.Linear(d_model * 2,
vocab_size)
            )
            for _ in range(num_streams)
        ])

        # Coordination head (produces
big_boat_summary)
        self.coordination_head =
nn.Sequential(
            nn.Linear(d_model * num_streams,
d_model * 2),
            nn.GELU(),
            nn.Linear(d_model * 2, d_model),
            nn.Linear(d_model, 256)  #
Fixed-size coordination vector
        )

        # Latency prediction head
        self.latency_head = nn.Sequential(
```

```
            nn.Linear(d_model, 64),
            nn.ReLU(),
            nn.Linear(64, 3)  #
[thinking_time, speaking_time, listening_time]
        )

    def forward(self, input_ids: torch.Tensor,
stream_ids: torch.Tensor,
                attention_mask: torch.Tensor =
None,
                is_training: bool = True) ->
Dict:
        """
        Forward pass with parallel streams

        Args:
            input_ids: (batch, seq_len) token
IDs
            stream_ids: (batch, seq_len) which
stream each token belongs to
            attention_mask: (batch, seq_len)
attention mask
            is_training: whether to use
training mode (parallel) or inference
(sequential)
        """
        batch_size, seq_len = input_ids.shape
```

```
        # Get token embeddings
        token_embeds =
self.token_embedding(input_ids)  # (batch,
seq_len, d_model)

        # Add stream embeddings
        stream_embeds =
self.stream_embeddings(stream_ids)  # (batch,
seq_len, d_model)
        x = token_embeds + stream_embeds

        if is_training:
            # PARALLEL PROCESSING MODE
(Training)
            # Each stream processes its tokens
in parallel

            # Group tokens by stream
            stream_outputs = []
            stream_attentions = []

            for stream_idx in
range(self.num_streams):
                # Mask for this stream's
tokens
                stream_mask = (stream_ids ==
```

```
stream_idx).unsqueeze(-1)  # (batch, seq_len,
1)
                stream_tokens = x *
stream_mask

                # Apply stream-specific
attention
                stream_tokens_flat =
stream_tokens.view(batch_size * seq_len,
self.d_model)
                stream_mask_flat =
stream_mask.view(batch_size * seq_len, 1)

                # Only process non-zero tokens
(efficient)
                non_zero_mask =
(stream_mask_flat.squeeze() > 0)
                if non_zero_mask.any():
                    stream_tokens_nonzero =
stream_tokens_flat[non_zero_mask].unsqueeze(0)

                    # Self-attention within
stream
                    attn_output, attn_weights
= self.stream_attentions[stream_idx](
                        stream_tokens_nonzero,
stream_tokens_nonzero, stream_tokens_nonzero
```

```
                    )

                    # Transform
                    transformed =
self.transformer(attn_output)

                    # Get output logits for
this stream
                    logits =
self.stream_outputs[stream_idx](transformed)


stream_outputs.append(logits.squeeze(0))

stream_attentions.append(attn_weights)
                else:

stream_outputs.append(None)

stream_attentions.append(None)

            # Cross-stream attention
(coordination)
            # Average embeddings per stream
            stream_representations = []
            for stream_idx in
range(self.num_streams):
```

```
                stream_mask = (stream_ids ==
stream_idx)
                if stream_mask.any():
                    stream_emb =
x[stream_mask].mean(dim=0, keepdim=True)

stream_representations.append(stream_emb)
                else:

stream_representations.append(torch.zeros(1,
self.d_model).to(x.device))

            # Stack stream representations
            stream_reps =
torch.cat(stream_representations, dim=1)  #
(batch, num_streams * d_model)

            # Coordination vector
            coordination =
self.coordination_head(stream_reps)

            # Latency prediction
            avg_embedding = x.mean(dim=1)  #
(batch, d_model)
            latency_pred =
self.latency_head(avg_embedding)
```

```
            return {
                "stream_logits":
stream_outputs,
                "stream_attentions":
stream_attentions,
                "coordination_vector":
coordination,
                "latency_prediction":
latency_pred
            }

        else:
            # SEQUENTIAL MODE (Inference - but
with overlapping)
            # Simulates real-time processing
where new tokens arrive during generation

            outputs = []
            coordination_vectors = []

            # Process in chunks with overlap
            chunk_size = 64
            overlap = 16

            for start_idx in range(0, seq_len,
chunk_size - overlap):
                end_idx = min(start_idx +
```

```
chunk_size, seq_len)

                chunk = x[:,
start_idx:end_idx, :]

                # Process chunk
                chunk_output =
self.transformer(chunk)

                # Get coordination for this
chunk
                chunk_coord =
self.coordination_head(

chunk_output.mean(dim=1).unsqueeze(1)
                )

                outputs.append(chunk_output)

coordination_vectors.append(chunk_coord)

            # Combine outputs
            combined = torch.cat(outputs,
dim=1)
            final_coordination =
torch.stack(coordination_vectors).mean(dim=0)
```

```
            # Get logits for each stream
            stream_logits = []
            for stream_idx in
range(self.num_streams):
                stream_mask = (stream_ids ==
stream_idx).unsqueeze(-1)
                stream_tokens = combined *
stream_mask

                # Only get logits where mask
is non-zero
                if stream_mask.any():
                    non_zero_indices =
stream_mask.squeeze().nonzero(as_tuple=True)
                    stream_nonzero =
stream_tokens[non_zero_indices]
                    logits =
self.stream_outputs[stream_idx](stream_nonzero
)

                    # Place back
                    full_logits =
torch.zeros_like(combined[...,
:self.stream_outputs[0][-1].out_features])

full_logits[non_zero_indices] = logits
```

```
stream_logits.append(full_logits)
                else:
                    stream_logits.append(None)

            return {
                "stream_logits":
stream_logits,
                "coordination_vector":
final_coordination,
                "latency_prediction":
self.latency_head(combined.mean(dim=1))
            }

class RealtimeTrainingPipeline:
    """Complete training pipeline for
real-time cognition"""

    def __init__(self, model, dataset,
num_epochs=10, batch_size=4):
        self.model = model
        self.dataset = dataset
        self.num_epochs = num_epochs
        self.batch_size = batch_size

        # Multiple loss functions
        self.loss_fns = {
            "stream_prediction":
```

```
nn.CrossEntropyLoss(),
            "coordination_mse": nn.MSELoss(),
            "latency_mae": nn.L1Loss(),
            "interruption_handling":
nn.BCEWithLogitsLoss()
        }

        # Curriculum schedule
        self.curriculum = [
            {"phase": 1, "name": "Basic
Parallelism", "difficulty": 0.2, "epochs": 2},
            {"phase": 2, "name":
"Interruptions", "difficulty": 0.4, "epochs":
2},
            {"phase": 3, "name": "Latency
Constraints", "difficulty": 0.6, "epochs": 2},
            {"phase": 4, "name": "Full
Coordination", "difficulty": 0.8, "epochs":
2},
            {"phase": 5, "name": "Real-time
Simulation", "difficulty": 1.0, "epochs": 2}
        ]

    def train_phase(self, phase_config: Dict,
optimizer, scheduler):
        """Train on a specific phase of the
curriculum"""
```

```
        print(f"\n{'='*60}")
        print(f"Starting Phase
{phase_config['phase']}:
{phase_config['name']}")
        print(f"Difficulty:
{phase_config['difficulty']}")
        print(f"{'='*60}")

        phase_losses = []

        for epoch in
range(phase_config["epochs"]):
            epoch_loss = 0
            num_batches = 0

            # Generate batches for this phase
            for batch_idx in range(100):  #
Simulate 100 batches per epoch
                batch =
self._generate_batch(phase_config["difficulty"
])

                # Forward pass
                outputs = self.model(
                    batch["input_ids"],
                    batch["stream_ids"],
```

```
attention_mask=batch["attention_mask"],
                    is_training=True
                )

                # Calculate multi-task loss
                total_loss = 0

                # Stream prediction loss
                for stream_idx, logits in
enumerate(outputs["stream_logits"]):
                    if logits is not None:
                        targets =
batch["stream_targets"][stream_idx]
                        loss =
self.loss_fns["stream_prediction"](
                            logits.view(-1,
logits.size(-1)),
                            targets.view(-1)
                        )
                        total_loss += loss

                # Coordination loss
                coord_loss =
self.loss_fns["coordination_mse"](

outputs["coordination_vector"],
```

```
batch["coordination_target"]
                )
                total_loss += coord_loss * 0.5

                # Latency prediction loss
                latency_loss =
self.loss_fns["latency_mae"](

outputs["latency_prediction"],
                    batch["latency_target"]
                )
                total_loss += latency_loss *
0.3

                # Interruption handling loss
(if present)
                if "interruption_target" in
batch:
                    # Simulate interruption
handling score
                    interruption_score =
torch.sigmoid(

outputs["coordination_vector"].mean()
                    )
                    inter_loss =
self.loss_fns["interruption_handling"](
```

```
                        interruption_score,

batch["interruption_target"]
                    )
                    total_loss += inter_loss *
0.2

                # Backward pass
                optimizer.zero_grad()
                total_loss.backward()

                # Gradient clipping for
stability

torch.nn.utils.clip_grad_norm_(self.model.para
meters(), 1.0)

                optimizer.step()

                epoch_loss +=
total_loss.item()
                num_batches += 1

                if batch_idx % 20 == 0:
                    print(f"  Batch
{batch_idx}: Loss = {total_loss.item():.4f}")
```

```
            avg_epoch_loss = epoch_loss /
num_batches

phase_losses.append(avg_epoch_loss)

            print(f"Epoch
{epoch+1}/{phase_config['epochs']}: Avg Loss =
{avg_epoch_loss:.4f}")

        return phase_losses

    def _generate_batch(self, difficulty:
float) -> Dict:
        """Generate a batch with specified
difficulty level"""
        batch_examples = []

        for _ in range(self.batch_size):
            example =
self.dataset.generate_example()

            # Adjust difficulty
            if difficulty < 0.3:
                # Easy: no interruptions,
simple streams
                example["events"] = []
            elif difficulty < 0.6:
```

```
                # Medium: some interruptions
                example["events"] = [e for e
in example["events"]
                                     if
e["type"] != "parallel_inputs"]
            # Hard: everything included

            batch_examples.append(example)

        # Convert to tensors
        return
self._collate_batch(batch_examples)

    def _collate_batch(self, examples: List)
-> Dict:
        """Collate batch examples into
tensors"""
        # Simplified collation
        max_len =
max(len(ex["dialogue"]["tokens"]) for ex in
examples)

        batch_size = len(examples)
        input_ids = torch.zeros(batch_size,
max_len, dtype=torch.long)
        stream_ids = torch.zeros(batch_size,
max_len, dtype=torch.long)
```

```
        attention_mask =
torch.zeros(batch_size, max_len,
dtype=torch.bool)

        # Stream targets (per token)
        stream_targets = []
        for stream_idx in
range(self.dataset.num_streams):

stream_targets.append(torch.zeros(batch_size,
max_len, dtype=torch.long))

        # Coordination target (dummy for now)
        coordination_target =
torch.randn(batch_size, 256)

        # Latency target (simulated)
        latency_target =
torch.randn(batch_size, 3) * 0.5 + 1.0  # ~1
second average

        # Fill tensors
        for i, example in enumerate(examples):
            tokens =
example["dialogue"]["tokens"]
            length = len(tokens)
```

```
            input_ids[i, :length] =
torch.tensor(tokens)
            attention_mask[i, :length] = True

            # Assign streams (simplified:
round-robin)
            for j in range(length):
                stream_ids[i, j] = j %
self.dataset.num_streams

            # Stream targets (next token
prediction)
            for stream_idx in
range(self.dataset.num_streams):
                stream_mask = (stream_ids[i]
== stream_idx)
                if stream_mask.any():
                    # Simple target: predict
next token in sequence
                    stream_target =
input_ids[i].roll(-1, dims=0)

stream_targets[stream_idx][i] = stream_target

        return {
            "input_ids": input_ids,
            "stream_ids": stream_ids,
```

```
            "attention_mask": attention_mask,
            "stream_targets": stream_targets,
            "coordination_target":
coordination_target,
            "latency_target": latency_target
        }

    def train(self):
        """Full curriculum training"""

        optimizer = torch.optim.AdamW(
            self.model.parameters(),
            lr=1e-4,
            weight_decay=0.01
        )

        scheduler =
torch.optim.lr_scheduler.CosineAnnealingWarmRe
starts(
            optimizer,
            T_0=10,
            T_mult=2
        )

        all_losses = []

        for phase in self.curriculum:
```

```
            phase_losses =
self.train_phase(phase, optimizer, scheduler)
            all_losses.extend(phase_losses)

            # Update scheduler
            scheduler.step()

        return all_losses

class RealtimeSimulator:
    """Simulate real-time interaction for
evaluation"""

    def __init__(self, model, tokenizer):
        self.model = model
        self.tokenizer = tokenizer
        self.model.eval()

        # Real-time metrics
        self.metrics = {
            "interruption_recovery_time": [],
            "parallel_processing_efficiency":
[],
            "context_integration_success": [],
            "latency_violations": []
        }
```

```
    def simulate_conversation(self,
num_turns=10):
        """Simulate a real-time conversation
with interruptions"""

        print("\n" + "="*60)
        print("REAL-TIME SIMULATION STARTING")
        print("="*60)

        context = []
        active_streams = set()

        for turn in range(num_turns):
            print(f"\n--- Turn {turn+1} ---")

            # Simulate user input (arrives
while model might be thinking)
            user_input =
self._generate_user_input(turn)
            print(f"User: {user_input}")

            # Check for interruption
            if np.random.random() < 0.3 and
turn > 0:
                print("  [INTERRUPTION! User
speaks again during processing]")
                interruption =
```

```
self._generate_interruption()
                print(f"  User (interrupting):
{interruption}")

                # Measure interruption
recovery
                start_time = time.time()
                recovery_success =
self._handle_interruption(
                    context, user_input,
interruption
                )
                recovery_time = time.time() -
start_time


self.metrics["interruption_recovery_time"].app
end(recovery_time)
                print(f"  Recovery time:
{recovery_time:.2f}s")

            # Simulate parallel streams
activating
            streams_to_activate =
np.random.choice(

list(self.model.stream_embeddings.num_embeddin
```

```
gs),
                size=np.random.randint(2, 4),
                replace=False
            )

            print(f"  Active streams:
{streams_to_activate}")

            # Process with overlapping context
            # Simulate new info arriving
mid-processing
            if np.random.random() < 0.4:
                print("  [New context arriving
during processing...]")
                new_context =
self._generate_new_context()

                # Test context integration
                integration_start =
time.time()
                success =
self._integrate_new_context(context,
new_context)
                integration_time = time.time()
- integration_start
```

```
self.metrics["context_integration_success"].ap
pend(
                    1.0 if success else 0.0
                )

            # Check latency
            processing_start = time.time()

            with torch.no_grad():
                # Tokenize input
                inputs =
self.tokenizer(user_input,
return_tensors="pt")

                # Add stream information
                stream_ids =
torch.zeros_like(inputs["input_ids"])
                for i, stream_id in
enumerate(streams_to_activate[:inputs["input_i
ds"].size(1)]):
                    stream_ids[0, i] =
stream_id

                # Forward pass (simulating
real-time with chunking)
                outputs = self.model(
                    inputs["input_ids"],
```

```
                    stream_ids,
                    is_training=False
                )

            processing_time = time.time() -
processing_start

            # Human latency parity: 2-3
seconds is acceptable
            if processing_time > 3.0:
                print(f"  [LATENCY VIOLATION:
{processing_time:.2f}s]")

self.metrics["latency_violations"].append(proc
essing_time)

            # Generate response (simplified)
            response =
self._generate_response(outputs, context)
            print(f"Assistant:
{response[:100]}...")

            # Update context
            context.append({"user":
user_input, "assistant": response})

            # Brief pause to simulate thinking
```

```
            if np.random.random() < 0.5:
                think_time =
np.random.uniform(0.5, 2.0)
                print(f"  [Thinking for
{think_time:.1f}s...]")
                time.sleep(min(think_time,
0.1))  # Cap for simulation

        # Print metrics
        self._print_metrics()

    def _handle_interruption(self, context,
original_input, interruption):
        """Simulate interruption handling"""
        # In real model, this would merge
contexts
        return np.random.random() > 0.2  # 80%
success rate

    def _integrate_new_context(self, context,
new_context):
        """Simulate new context integration"""
        return np.random.random() > 0.3  # 70%
success rate

    def _print_metrics(self):
        """Print simulation metrics"""
```

```
        print("\n" + "="*60)
        print("SIMULATION METRICS")
        print("="*60)

        for metric_name, values in
self.metrics.items():
            if values:
                avg = np.mean(values)
                std = np.std(values)
                print(f"{metric_name}:
{avg:.3f} ± {std:.3f}")
            else:
                print(f"{metric_name}: No
data")

        # Calculate overall real-time score
        if self.metrics["latency_violations"]:
            latency_score = 1.0 / (1.0 +
np.mean(self.metrics["latency_violations"]))
        else:
            latency_score = 1.0

        if
self.metrics["interruption_recovery_time"]:
            recovery_score = 1.0 / (1.0 +
np.mean(self.metrics["interruption_recovery_ti
me"]))
```

```
        else:
            recovery_score = 1.0

        if
self.metrics["context_integration_success"]:
            context_score =
np.mean(self.metrics["context_integration_succ
ess"])
        else:
            context_score = 1.0

        overall_score = (latency_score * 0.4 +

                         recovery_score * 0.3 +

                         context_score * 0.3)

        print(f"\nOverall Real-time Score:
{overall_score:.3f}/1.0")

        if overall_score > 0.7:
            print("☑ Model ready for
real-time deployment")
        elif overall_score > 0.5:
            print("⚠  Model needs improvement
for real-time use")
        else:
```

```
            print("❌ Model not suitable for
real-time interaction")

# Main training execution
if __name__ == "__main__":
    print("Building real-time interaction
training pipeline...")

    # Initialize components
    dataset =
RealtimeInteractionDataset(num_streams=4)
    model = ParallelCognitionTransformer(
        vocab_size=50257,
        d_model=768,
        n_heads=12,
        n_layers=6,  # Smaller for faster
training
        num_streams=4
    )

    # Training pipeline
    pipeline = RealtimeTrainingPipeline(
        model=model,
        dataset=dataset,
        num_epochs=10,
        batch_size=4
    )
```

```
    # Train
    print("\nStarting curriculum training...")
    losses = pipeline.train()

    # Save model
    torch.save(model.state_dict(),
"realtime_cognition_model.pt")
    print("Model saved to
realtime_cognition_model.pt")

    # Simulate real-time interaction
    print("\n" + "="*60)
    print("Starting real-time simulation...")
    print("="*60)

    # Load tokenizer (for simulation)
    tokenizer =
AutoTokenizer.from_pretrained("gpt2")

    simulator = RealtimeSimulator(model,
tokenizer)

simulator.simulate_conversation(num_turns=8)
```